

Databricks 14 Days AI Challenge

Day 2: Apache spark fundamentals (10/01/26)

1. Spark architecture (driver, execution, DAG)?

Basics:

RDD: Resilient Distributed dataset

DAG: Directed Acyclic Graph

Resilient Distributed dataset

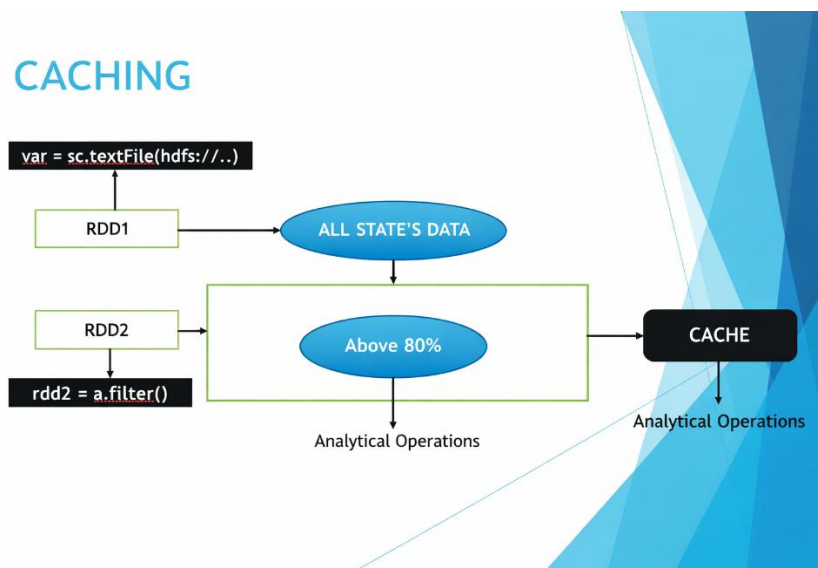
- a. Fault tolerant distributed dataset
- b. Lazy evaluation

Spark does nothing in clusters until you ask for a result.

You can write 10 lines of transformations and Spark will just sit there, planning.

No data is read. No computation happens. Like I have wrote some transformation below, it will not do anything until I ask for it.

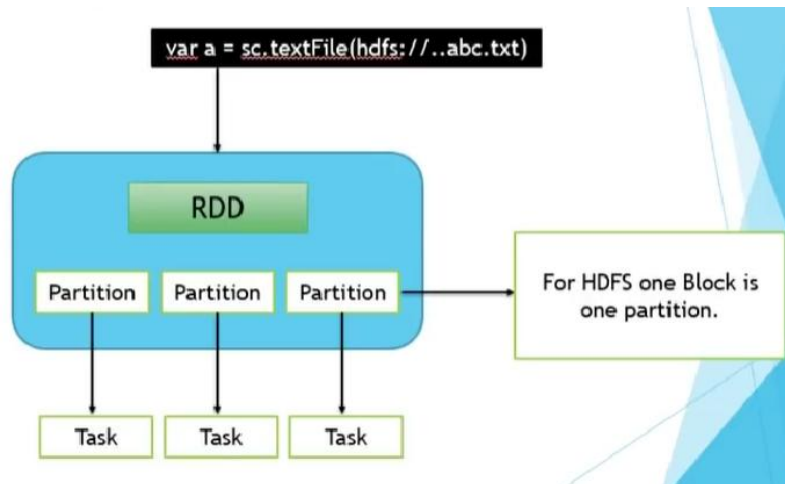
c. Caching



First we apply rdd1 on one dataset and then afterwards if you want to apply it on other dataset or any problem, the analytical approach is saved in CACHE and you can use it any other dataset

- d. In memory computation
- e. Immutability and Partitioning

- Data is split up into partitions.
- Partition size depends on data source you are using.
- For HDFS (Hadoop distributed file system) one Block is one Partition.
- Single partition → Single Task



Transformation:

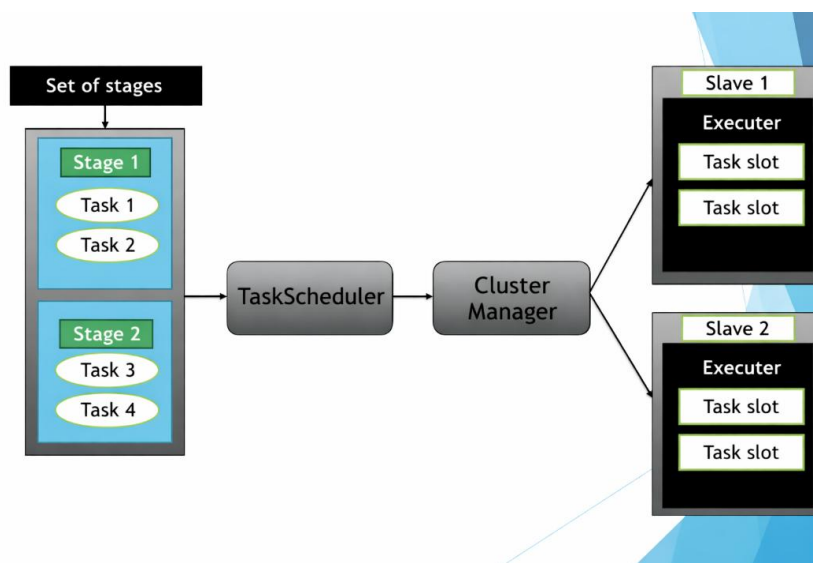
```
var a = sc.textFile("")
var b = a.filter(...)
var c = b.distinct(...)
```

Transformations are of two types:

Narrow transformations (map, filter, union, sample...)

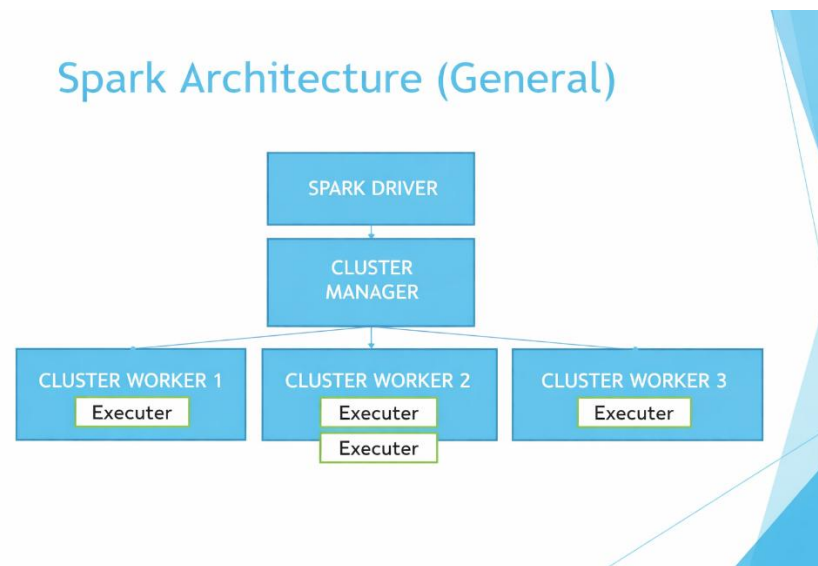
Wide transformations (intersection, join...)

Actions: collect(), show(), count(), collect(), write(), take()



Spark splits your job into stages, then into tasks. The Task Scheduler sends those tasks to the Cluster Manager, which assigns them to machines (slaves). Each machine runs tasks using executors with multiple task slots, so many tasks run in parallel and the job finishes faster.

SPARK ARCHITECTURE



Working of the Spark architecture

When we submit the code to spark driver, there are some components in the spark which converts code into DAG and then this DAG is given to DAG scheduler and then the output came from DAG scheduler which is comprises in stages and then given to task scheduler. After that task scheduler, pick up the task and negotiate with the cluster manager and then cluster manager search the data to perform that individual task and send the task to that data to execute the task and finally the task are executed in the executor, in which executor read, write and the code and finish the task and sends back to driver after completion and then after completing the task, the resources which are allocated for the execution of the task, are deallocated.

❖ Driver

The **driver** is the brain.

What it does:

- runs your Spark application's main code
- creates the Spark Session
- talks to the cluster manager
- builds the execution plan (DAG)
- sends work to executors
- collects results back

❖ Executors

Executors are the workers. What they do:

- execute tasks assigned by the driver
- process data partitions
- store data in memory or disk (caching)
- return results to the driver

❖ DAG (Directed Acyclic Graph)

This is how Spark understands your code.

When you write Spark code, Spark doesn't execute it line by line.

Instead:

- it records **transformations** (map, filter, join)
- builds a **DAG** showing how data flows
- waits until an **action** (count, show, write) is called
- then optimizes and executes the DAG

Key idea:

Transformations are lazy. Actions trigger execution.

2. Data frames vs RDDs

RDDs are low-level distributed collections with no schema or optimization, while Data Frames are schema-aware and allow Spark to optimize execution using Catalyst, making them faster and easier to use for most analytics workloads.

3. Notebook magic commands (%sql, %python, %fs, %sh)

Databricks magic commands allow switching languages at the cell level.

%python runs Spark logic,

%sql is used for querying and validating data,

%fs is used to interact with the underlying file system,

%md — Markdown, Used to write documentation inside notebooks,

%run - Executes another notebook inside the current one,

%sh Runs Linux shell commands on the cluster.

%fs	%fs ls /mnt/raw/sales/
%run	%run /Shared/utis/transformations
%sh	%sh ls -lh